

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 625 — Artificial Intelligence

Search, logic, and decision-making — the AI that ships in game engines and runs without a GPU

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 7 (with CSCE 753 and CSCE 642)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 669 Computational Optimization is helpful for Modules 06 and 10; probability

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: An agent that solves a real problem, with the approach justified against the alternatives, the search or inference instrumented, and an honest account of what it cannot do

Course description

This is the AI that is not a neural network, and the ordering of this semester is deliberate: CSCE 753 and CSCE 642 cover the learned approaches, and **this course covers the enormous amount of working AI that predates them and has not been displaced.**

Every game that ships contains the contents of this course. Pathfinding is Module 04. Enemy behaviour is Module 11. Procedural generation is Module 06. Tactical decision-making is Module 07 or Module 10. **None of it is learned, all of it is debuggable, and it runs in a millisecond on a CPU core** — which are the three properties a game engine actually requires.

The organising idea is that intelligence can be search. Modules 02 through 07 are all the same algorithm wearing different clothes — a frontier, a successor function, and a way of deciding what to expand next — and the differences between pathfinding, game-tree search, constraint solving, and planning are differences in the state space and the heuristic, not in the method.

The second idea is that a good heuristic is worth more than a faster computer. Module 03 makes this precise: an admissible heuristic that is twice as informative cuts the search exponentially, and **deriving heuristics automatically from relaxed problems (Module 07) is one of the field's genuinely beautiful results.**

The third is that an agent must act under uncertainty, which Modules 09 and 10 treat properly — and which connects directly to CSCE 642, where the same Markov decision process is solved by learning instead of by computation.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Formulate a problem as a state space and choose a search strategy.
- 2 State and verify admissibility and consistency, and design heuristics.
- 3 Implement production-quality pathfinding, including on navigation meshes.
- 4 Implement adversarial search with alpha-beta and Monte Carlo tree search.
- 5 Model and solve a constraint satisfaction problem, including for content generation.
- 6 Explain classical planning and derive heuristics from relaxations.
- 7 Use a SAT solver and explain what it is doing.
- 8 Build and query a Bayesian network, and explain conditional independence.
- 9 Solve a Markov decision process by value and policy iteration.
- 10 Choose and implement a game AI architecture and justify the choice.
- 11 State what an AI system can be trusted to do, and why.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Agents and Rationality	What makes a program an agent, and what makes an agent good?
2	Search	How do you systematically explore a space of possibilities?
3	Heuristic Search	How much is a good estimate worth?
4	Pathfinding in Practice	How does a game actually move a character across a level?
5	Adversarial Search	How do you plan against an opponent who is planning against you?
6	Constraint Satisfaction	What if you could prune before searching rather than while?
7	Classical Planning	What if the agent is told what actions do rather than which to take?
8	Logic and Satisfiability	What can you get from stating a problem as a logical formula?
9	Probabilistic Reasoning	How do you represent and update uncertain beliefs?
10	Decisions Under Uncertainty	How do you act well when actions have uncertain outcomes?
11	Game AI Architecture	How do you organise an agent's behaviour so a designer can work with it?
12	Multiple Agents	What happens when many agents act at once?

Wk	Module	Core question it answers
13	Choosing an Approach	Given a problem, which of these twelve modules applies?

Projects

Project 1 — A searching agent, instrumented (after Module 7)

Build an agent that solves a non-trivial problem by search or planning, and measure what the search actually does.

Requirements

- A **state space formulation** written out: states, actions, successor function, goal test, costs.
- **At least three search strategies compared** on the same problem, with nodes expanded reported for each.
- **At least two heuristics**, with admissibility argued or disproved for each.
- A **plot of nodes expanded against problem size** for each configuration.
- A **visualisation of the search** — what was expanded, in what order.
- **One problem instance your agent cannot solve**, with the reason identified as branching factor, depth, or heuristic weakness.

Done means

- **Nodes expanded reported, not just wall-clock time** — time confounds the algorithm with the implementation.
- **The admissibility arguments correct**, including the one you disprove.
- **The better heuristic demonstrably dominating**, with the node-count ratio stated.
- **The failure case explained in terms of the state space**, not described as slowness.

Project 2 — An agent that decides (after Module 12)

Build an agent that acts under uncertainty or against an opponent, in a real-time loop, and justify the architecture.

Requirements

- A **real-time loop with a per-decision time budget** stated and measured.
- **An architecture chosen from Module 11's options**, with the alternatives considered and rejected in writing.
- **Either adversarial search (Module 05) or an MDP solved (Module 10), with the state space size reported.**
- **Behaviour that is debuggable** — a way to ask the agent why it did what it did.
- A **comparison against a trivial baseline policy**. CSCE 633's discipline, unchanged.
- **Latency at the 99th percentile**, not the mean.

Done means

- **The time budget met at p99**, with the measurement shown.
- **The architecture choice justified against at least two alternatives**, in terms of this problem.
- **The agent beats the trivial baseline**, measurably, or you explain why it does not.
- **A written account of what the agent cannot do** and what would be needed — which is the graded part.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Russell & Norvig — Artificial Intelligence: A Modern Approach, 4th edition	The primary source for this course. The site hosts free pseudocode for every algorithm, free Python implementations, and the figures. Library copy for the text; everything algorithmic is free.
Berkeley CS188 — Introduction to Artificial Intelligence (free lectures, notes, and the Pacman projects)	The best free AI course available, and the Pacman assignments are outstanding. Modules 02–05 and 10 follow it closely.
Millington — AI for Games, 3rd edition	The reference for Modules 04 and 11 — the engineering detail that academic treatments omit. The free <i>Game AI Pro</i> volumes at this link are chapters by shipping practitioners and are excellent.
Ghallab, Nau & Traverso — Automated Planning and Acting (free draft)	Module 07's reference , with the relaxation-based heuristics developed properly.
Biere, Heule, van Maaren & Walsh — Handbook of Satisfiability (chapters free)	Module 08's CDCL material , and the competition results that show how far solvers have come.
Sutton & Barto — Reinforcement Learning, 2nd edition (free PDF)	Chapters 3 and 4 are Module 10 , and the rest is CSCE 642 — so the two courses share one free book, which makes the connection easy to follow.

Tooling

- **Python for the algorithms, and a profiler.** Search performance is dominated by the successor function and the frontier data structure, and both are easy to get wrong in ways only a profiler reveals.
- **A visualiser you write yourself.** Watching a search expand is worth more than any amount of reading, and a hundred-line grid renderer will serve the whole course.
- **A SAT solver:** MiniSat, CaDiCaL, or PySAT. **Module 08 is much more convincing when you watch a solver dispatch a problem you could not.**
- **A game engine or a simple simulation loop** for Modules 04 and 11. **Godot is free and adequate;** so is a Pygame loop, and the point is the agent rather than the renderer.
- **pgmpy or similar for Module 09,** after implementing variable elimination yourself once.
- **Node counts instrumented everywhere.** This course's measurements are nodes expanded and time per decision, and neither is visible unless you count it.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.